

Container Adapter

容器适配器

栈

STL [栈](#)(`std::stack`) 是一种后进先出 (Last In, First Out) 的容器适配器，仅支持查询或删除最后一个加入的元素（栈顶元素），不支持随机访问，且为了保证数据的严格有序性，不支持迭代器。

头文件

```
1 #include <stack>
```

定义

```
1 std::stack<TypeName> s; // 使用默认底层容器 deque, 数据类型为 TypeName
2 std::stack<TypeName, Container> s; // 使用 Container 作为底层容器
3 std::stack<TypeName> s2(s1); // 将 s1 复制一份用于构造 s2
```

成员函数

以下所有函数均为常数复杂度

- `top()` 访问栈顶元素（如果栈为空，此处会出错）
- `push(x)` 向栈中插入元素 `x`
- `pop()` 删除栈顶元素
- `size()` 查询容器中的元素数量
- `empty()` 询问容器是否为空

简单示例

```
1 std::stack<int> s1;
2 s1.push(2);
3 s1.push(1);
4 std::stack<int> s2(s1);
5 s1.pop();
6 std::cout << s1.size() << " " << s2.size() << std::endl; // 1 2
7 std::cout << s1.top() << " " << s2.top() << std::endl; // 2 1
8 s1.pop();
9 std::cout << s1.empty() << " " << s2.empty() << std::endl; // 1 0
```

队列

STL [队列](#)(`std::queue`) 是一种先进先出 (First In, First Out) 的容器适配器，仅支持查询或删除第一个加入的元素（队首元素），不支持随机访问，且为了保证数据的严格有序性，不支持迭代器。

头文件

```
1 #include <queue>
```

定义

```
1 std::queue<TypeName> q; // 使用默认底层容器 deque, 数据类型为 TypeName
2 std::queue<TypeName, Container> q; // 使用 Container 作为底层容器
3
4 std::queue<TypeName> q2(q1); // 将 s1 复制一份用于构造 q2
```

成员函数

以下所有函数均为常数复杂度

- `front()` 访问队首元素（如果队列为空，此处会出错）
- `push(x)` 向队列中插入元素 `x`
- `pop()` 删除队首元素
- `size()` 查询容器中的元素数量
- `empty()` 询问容器是否为空

简单示例

```
1 std::queue<int> q1;
2 q1.push(2);
3 q1.push(1);
4 std::queue<int> q2(q1);
5 q1.pop();
6 std::cout << q1.size() << " " << q2.size() << std::endl; // 1 2
7 std::cout << q1.front() << " " << q2.front() << std::endl; // 1 2
8 q1.pop();
9 std::cout << q1.empty() << " " << q2.empty() << std::endl; // 1 0
```

优先队列

优先队列 `std::priority_queue` 是一种 [堆](#)，一般为 [二叉堆](#)。

头文件

```
1 #include <queue>
```

定义

```

1 std::priority_queue<TypeName> q;           // 数据类型为 TypeName
2 std::priority_queue<TypeName, Container> q; // 使用 Container 作为底层容器
3 std::priority_queue<TypeName, Container, Compare> q;
4 // 使用 Container 作为底层容器, 使用 Compare 作为比较类型
5
6 // 默认使用底层容器 vector
7 // 比较类型 less<LTTypeName> (此时为它的 top() 返回为最大值)
8 // 若希望 top() 返回最小值, 可令比较类型为 greater<LTTypeName>
9 // 注意: 不可跳过 Container 直接传入 Compare
10
11 // 从 C++11 开始, 如果使用 lambda 函数自定义 Compare
12 // 则需要将其作为构造函数的参数代入, 如:
13 auto cmp = [](const std::pair<int, int> &l, const std::pair<int, int> &r) {
14     return l.second < r.second;
15 };
16 std::priority_queue<std::pair<int, int>, std::vector<std::pair<int, int>>,
17                     decltype(cmp)>
18     pq(cmp);

```

成员函数

以下所有函数均为常数复杂度

- `top()` 访问堆顶元素 (此时优先队列不能为空)
- `empty()` 询问容器是否为空
- `size()` 查询容器中的元素数量

以下所有函数均为对数复杂度

- `push(x)` 插入元素, 并对底层容器排序
- `pop()` 删除堆顶元素 (此时优先队列不能为空)

简单示例

```

1 std::priority_queue<int> q1;
2 std::priority_queue<int, std::vector<int>> q2;
3 // C++11 后空格可省略
4 std::priority_queue<int, std::deque<int>, std::greater<int>> q3;
5 // q3 为小根堆
6 for (int i = 1; i <= 5; i++) q1.push(i);
7 // q1 中元素 : [1, 2, 3, 4, 5]
8 std::cout << q1.top() << std::endl;
9 // 输出结果 : 5
10 q1.pop();
11 // 堆中元素 : [1, 2, 3, 4]
12 std::cout << q1.size() << std::endl;
13 // 输出结果 : 4
14 for (int i = 1; i <= 5; i++) q3.push(i);
15 // q3 中元素 : [1, 2, 3, 4, 5]
16 std::cout << q3.top() << std::endl;
17 // 输出结果 : 1

```

本页面最近更新: 2024/10/9 22:38:42, [更新历史](#)

发现错误? 想一起完善? [在 GitHub 上编辑此页!](#)

本页面贡献者: [Early0v0](#), [ksyx](#), [akakw1](#), [aofall](#), [Enter-tainer](#), [Haohu Shen](#), [lr1d](#), [Lewy Zeng](#), [ouuan](#),

[Suyun514](#), [Tiphereth-A](#), [william-song-shy](#), [Xeonacid](#)

本页面的全部内容在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用